

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP LỚN
MÔN ĐỒ ÁN THIẾT KẾ MẠCH ĐIỆN TỬ
ĐỀ TÀI
THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG
TRẠM SẠC XE ĐIỆN CÓ GIÁM SÁT ĐIỆN
NĂNG VÀ THANH TOÁN TRỰC TUYẾN

Giảng viên hướng dẫn: ThS. Trương Minh Đức

Sinh viên thực hiện: Nguyễn Văn Thiện – B22DCDT308

Dương Thành Đạt – B22DCDT075

Mục Lục

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	4
1.1. Đặt vấn đề	4
1.2. Mục tiêu đề tài	4
1.3. Đối tượng và phạm vi nghiên cứu	4
1.4. Phương pháp thực hiện	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	5
2.1. Vi điều khiển ESP32	5
2.2. Framework phát triển ESP-IDF	6
2.3. Cảm biến điện năng PZEM-004T	7
2.4. Module Relay điều khiển công suất	8
2.5. Hệ sinh thái Cloud và IoT (Firebase)	8
2.6. Hệ thống Backend và Cổng thanh toán	9
2.7. Cơ chế đồng bộ thời gian thực qua giao thức NTP	9
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	10
3.1. Phân tích yêu cầu hệ thống	10
3.1.1. Yêu cầu chức năng	10
3.1.2. Yêu cầu phi chức năng	10
3.2. Kiến trúc tổng thể hệ thống	10
3.3. Thiết kế State Machine (FSM)	11
3.4. Thiết kế phần cứng (Hardware Design)	12
3.4.1. Sơ đồ nguyên lý (Schematic Design)	12
3.4.2. Cấu hình các chân GPIO	12
3.4.3. Thiết kế mạch in (PCB Layout)	13
3.4.4. Thi công và hoàn thiện	14
3.5. Thiết kế phần mềm và Cơ sở dữ liệu	14
3.5.1. Phân bổ đa nhiệm trên ESP-IDF	14
3.5.2. Cấu trúc dữ liệu Firebase	15
3.5.3. Quy trình quyết toán 2 pha (2-Phase Settlement)	15

CHƯƠNG 4: XÂY DỰNG HỆ THỐNG.....	16
4.1. Thiết kế phần cứng.....	16
4.2. Thiết kế Firmware trên ESP32 (ESP-IDF)	16
4.2.1. Module Main & Config (Quản trị hệ thống).....	16
4.2.2. Module Globals & Types (Cấu trúc dữ liệu)	16
4.2.3. Module Cloud(Giao tiếp đám mây).....	17
4.2.4. Module Sensor & Safety (Giám sát & Bảo vệ).....	17
4.2.5. Module Relay, UI & NVS (Ngoại vi & Lưu trữ).....	17
4.3. Web Dashboard và Backend Server	17
4.3.1. Giao diện Web App (React & Firebase SDK).....	17
4.3.2. Giao diện và các tính năng chính	18
4.3.3. Backend Server (Express & SePay Integration)	21
CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ	21
5.1. Kịch bản kiểm thử.....	21
5.2. Kết quả kiểm thử	21
5.3. Đánh giá hiệu năng và tính ổn định	22
5.3.1. Phản ứng thời gian thực (Real-time response)	22
5.3.2. Quản lý bộ nhớ và tài nguyên	22
5.3.3. Tính chính xác trong quyết toán tài chính.....	22
5.4. Đánh giá thực tế từ người dùng.....	23
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	23
6.1. Kết quả đạt được	23
6.2. Hạn chế của đề tài	23
6.3. Hướng phát triển.....	23
TÀI LIỆU THAM KHẢO	24

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Sự chuyển dịch từ phương tiện giao thông sử dụng động cơ đốt trong sang xe điện (EV) đang diễn ra mạnh mẽ trên toàn cầu và tại Việt Nam. Tuy nhiên, hạ tầng trạm sạc vẫn là một thách thức lớn. Việc triển khai các trạm sạc công cộng đòi hỏi chi phí đầu tư cao, trong khi các giải pháp sạc tại nhà hoặc văn phòng thường thiếu khả năng giám sát thông minh và cơ chế thanh toán minh bạch.

Hệ thống EVION ra đời nhằm giải quyết bài toán này bằng cách cung cấp một giải pháp trạm sạc IoT năng động. Thay vì xây dựng một hệ thống sạc phức tạp theo các tiêu chuẩn công nghiệp đắt đỏ, đề tài tập trung vào việc tạo ra một trạm cấp nguồn xoay chiều thông minh, cho phép sử dụng các bộ sạc đa năng có sẵn để sạc cho các loại xe điện thông dụng, đồng thời tích hợp quản lý năng lượng và thanh toán tự động qua môi trường Cloud.

1.2. Mục tiêu đề tài

Mục tiêu chính của đồ án là thiết kế và hiện thực hóa một hệ thống quản lý trạm sạc xe điện thông minh bao gồm:

- **Về phần cứng (Firmware):** Xây dựng bộ điều khiển trung tâm dựa trên chip ESP32 sử dụng framework chuyên nghiệp ESP-IDF v5.4.3. Hệ thống phải có khả năng đọc dữ liệu điện năng thời gian thực và điều khiển đóng ngắt relay an toàn.
- **Về phần mềm (Cloud & Web App):** Phát triển ứng dụng Web giúp người dùng tìm kiếm trạm, nạp tiền vào ví điện tử và theo dõi quá trình sạc.
- **Về quản lý và vận hành:** Tích hợp hệ thống thanh toán tự động qua SePay và cơ chế quyết toán hai pha (2-phase settlement) để đảm bảo tính chính xác về mặt tài chính.

1.3. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Hệ thống nhúng điều khiển thiết bị điện, giao thức truyền thông IoT (HTTP, REST API) và cơ sở dữ liệu thời gian thực.
- **Phạm vi:** Trạm sạc cấp nguồn xoay chiều (AC) cho các thiết bị sạc xe điện cá nhân. Hệ thống tập trung vào việc giám sát và quản lý từ xa thay vì đi sâu vào các giao thức sạc vật lý phức tạp.

1.4. Phương pháp thực hiện

Đề tài được thực hiện dựa trên sự kết hợp của các công nghệ hiện đại:

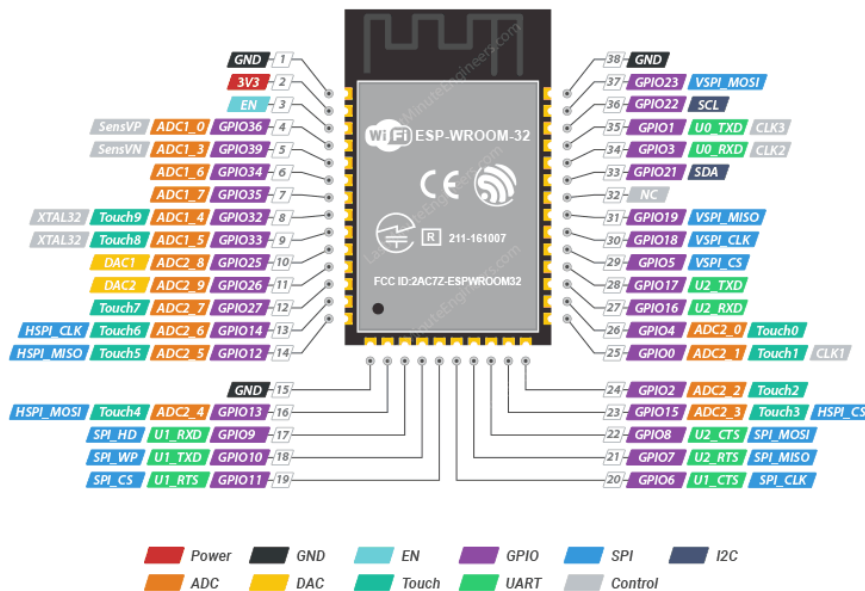
- **Thiết kế Firmware:** Sử dụng ngôn ngữ C trên nền tảng ESP-IDF, áp dụng cơ chế đa nhiệm (FreeRTOS) để tối ưu hóa việc giám sát an toàn và truyền thông.
- **Hạ tầng dữ liệu:** Sử dụng Firebase Realtime Database để đồng bộ trạng thái thiết bị và Firestore để quản lý thông tin người dùng, lịch sử giao dịch.
- **Phát triển Web & Server:** Sử dụng React (Vite) cho giao diện người dùng và Express Server (Node.js) làm trung gian xử lý các lệnh điều khiển và thanh toán.

Tiếp tục với nội dung báo cáo, dưới đây là **Chương 2** tập trung vào các nền tảng công nghệ cốt lõi. Chương này khẳng định tính chuyên nghiệp của đề tài thông qua việc sử dụng framework chuyên sâu và các giao thức công nghiệp.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Vi điều khiển ESP32

Trạm sạc sử dụng vi điều khiển trung tâm là **ESP32-WROOM-32D**, một hệ thống trên chip (SoC) tích hợp WiFi và Bluetooth công suất thấp. Đây là lựa chọn tối ưu cho các dự án IoT nhờ khả năng xử lý mạnh mẽ và hệ thống ngoại vi phong phú.



ESP32-WROOM-32 Pinout

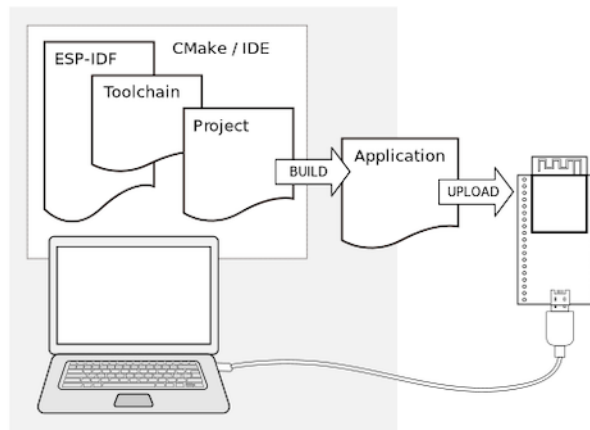
Thông số kỹ thuật chi tiết:

- **Vi xử lý:** Xtensa® Dual-Core 32-bit LX6, xung nhịp lên đến 240 MHz.
- **Bộ nhớ:** 520 KB SRAM nội bộ, tích hợp 8 MB Flash ngoài (thông qua giao tiếp SPI).

- **Kết nối không dây:**
 - WiFi: 802.11 b/g/n (tốc độ lên đến 150 Mbps).
 - Bluetooth: v4.2 BR/EDR và BLE (Bluetooth Low Energy).
- **Ngoại vi:** 34 chân GPIO, 12 kênh ADC 12-bit, 2 kênh DAC 8-bit, 3 cổng UART, 3 cổng SPI, 2 cổng I2C.
- **Nguồn điện:** Hoạt động ổn định ở mức 3.3V, dòng điện tiêu thụ thấp trong các chế độ Sleep.

Với kiến trúc Dual-core, ESP32 cho phép hệ thống vận hành song song: một nhân chuyên trách việc xử lý giao thức kết nối Cloud, nhân còn lại tập trung hoàn toàn vào việc giám sát an toàn và đọc cảm biến.

2.2. Framework phát triển ESP-IDF



Khác với các môi trường nghiệp dư, dự án này được phát triển trên ESP-IDF (Espressif IoT Development Framework), đây là framework chính thống và chuyên nghiệp nhất của nhà sản xuất.

- **Hệ điều hành thời gian thực (FreeRTOS):** ESP-IDF tích hợp sẵn FreeRTOS, cho phép lập trình theo tư duy đa nhiệm (multi-tasking). Các tác vụ (Tasks) được phân quyền ưu tiên (Priority) rõ rệt.
- **Task Scheduling và Task Pinning:** Cơ chế này cho phép "ghim" một tác vụ cụ thể vào một nhân CPU xác định (Core 0 hoặc Core 1). Điều này cực kỳ quan trọng trong trạm sạc để đảm bảo tác vụ bảo vệ quá dòng không bị trì hoãn bởi các hoạt động mạng.

- **Cấu hình chuyên sâu (SDK Configuration):** Cho phép tinh chỉnh sâu vào các thông số như ngưỡng bảo vệ điện áp thấp (Brownout detector), công suất phát WiFi (TX Power) để tối ưu hóa sự ổn định cho phần cứng.

2.3. Cảm biến điện năng PZEM-004T



Module PZEM-004T được sử dụng như một "đồng hồ đo điện thông minh" hỗ trợ giám sát toàn diện tải AC.

- **Thông số đo lường:**
 - Điện áp: 80 ~ 260 VAC (Sai số 0.5%).
 - Dòng điện: 0 ~ 100 A (Dòng khởi động thấp 0.01A).
 - Công suất: 0 ~ 23 kW.
 - Năng lượng: 0 ~ 9999 kWh.
 - Tần số: 45 ~ 65 Hz.
 - Hệ số công suất (PF): 0.00 ~ 1.00.
- **Giao thức giao tiếp (Modbus RTU):** PZEM-004T sử dụng chuẩn Modbus RTU trên nền UART. Mọi yêu cầu đọc dữ liệu đều phải đi kèm mã kiểm tra lỗi CRC-16 để đảm bảo dữ liệu không bị sai lệch do nhiễu từ dòng điện lớn trong trạm sạc gây ra.
- **Cách ly quang:** Module tích hợp bộ cách ly quang học để bảo vệ vi điều khiển ESP32 khỏi các xung điện áp cao từ lưới điện 220V.

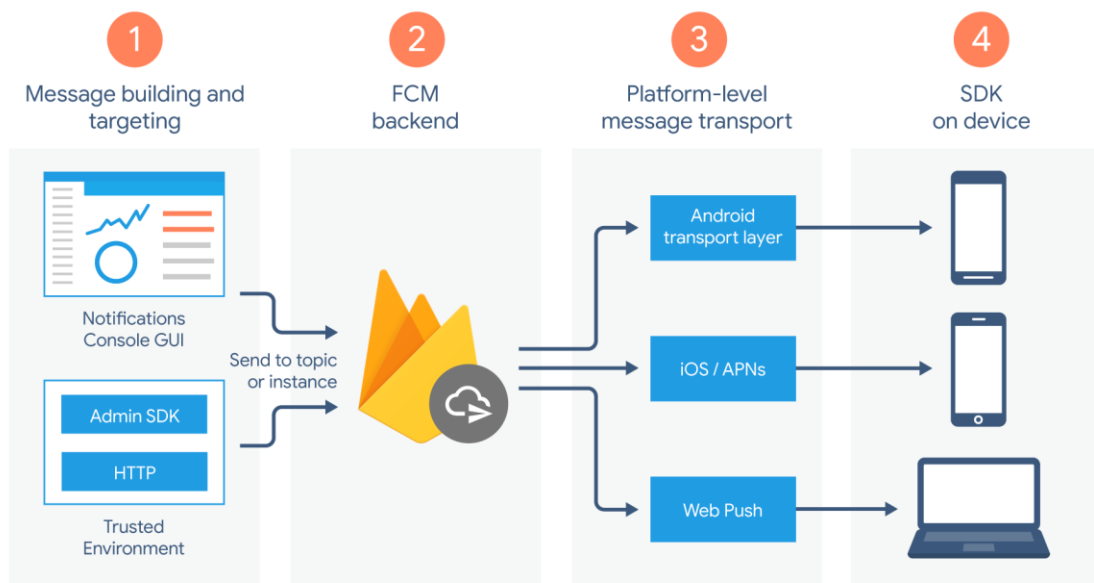
2.4. Module Relay điều khiển công suất



Relay đóng vai trò là "công tắc điện tử" thực thi lệnh đóng/ngắt nguồn cung cấp cho xe điện.

- **Nguyên lý hoạt động:** ESP32 kích mức logic cao (Active HIGH) qua chân GPIO32 để kích dẫn transistor/opto, cấp nguồn cho cuộn dây của Relay, từ đó hút tiếp điểm cơ khí để đóng mạch.
- **Đặc tính kỹ thuật:** Relay được chọn có khả năng chịu tải lên đến 30A/250VAC, phù hợp với các bộ sạc xe điện thông dụng hiện nay.
- **An toàn vận hành:** Trạng thái của Relay luôn được firmware giám sát thông qua hàm `gpio_get_level()`. Nếu phát hiện relay không đóng đúng lệnh, hệ thống sẽ ngay lập tức kích hoạt trạng thái báo lỗi (FAULT).

2.5. Hệ sinh thái Cloud và IoT (Firebase)



Hệ thống sử dụng nền tảng Cloud của Google để quản lý dữ liệu và điều khiển từ xa một cách tức thời.

- **Firebase Realtime Database (RTDB):** Đóng vai trò cốt lõi cho truyền thông thời gian thực. Dữ liệu Telemetry (V, A, W, kWh) được đẩy lên RTDB theo cấu trúc JSON mỗi 5 giây, giúp Web App hiển thị biểu đồ sạc mà không có độ trễ nhận thấy được.
- **Firebase Firestore:** Dùng để lưu trữ các dữ liệu có cấu trúc và cần tính bảo mật cao như: hồ sơ người dùng, số dư ví tiền, danh sách các trạm sạc và lịch sử giao dịch chi tiết.

2.6. Hệ thống Backend và Cổng thanh toán

Đây là lớp xử lý logic giúp trạm sạc trở thành một sản phẩm thương mại hoàn chỉnh.

- **Express API Server:** Chạy trên môi trường Node.js, xử lý các yêu cầu phức tạp như bắt đầu phiên sạc, kiểm tra số dư và thực hiện lệnh hoàn tiền (refund).
- **SePay Integration:** Tích hợp qua cơ chế Webhook. Khi người dùng quét mã QR và chuyển khoản thành công, SePay sẽ gửi thông tin giao dịch về Server. Hệ thống tự động bóc tách nội dung chuyển khoản để xác định định danh người dùng và cộng tiền vào ví điện tử ngay lập tức.
- **Xác thực và Bảo mật:** Sử dụng Firebase Auth để định danh người dùng và thuật toán ký HMAC-SHA256 để bảo vệ các báo cáo tài chính từ ESP32 gửi về Server, chống lại các hành vi gian lận năng lượng.

2.7. Cơ chế đồng bộ thời gian thực qua giao thức NTP

Trong hệ thống trạm sạc IoT, thời gian chính xác là yếu tố bắt buộc để quản lý nhật ký giao dịch và đối soát tài chính. Do ESP32 không có pin dự phòng cho đồng hồ thời gian thực (RTC) nội bộ, hệ thống sử dụng giao thức NTP (Network Time Protocol) để đồng bộ giờ từ internet.

- **Nguyên lý:** ESP32 sử dụng thư viện `esp_netif_sntp` (IDF 5.x) gửi yêu cầu đến các máy chủ thời gian như `pool.ntp.org` qua cổng UDP 123 ngay khi có kết nối WiFi.
- **Cơ chế cập nhật:** Hệ thống thực hiện đồng bộ ngay khi khởi động và tự động cập nhật lại sau mỗi 1 giờ để bù đắp sai số của thạch anh nội bộ.
- **Xử lý lỗi:** Nếu không thể kết nối server NTP, hệ thống sẽ tự động thử lại sau mỗi 60 giây để đảm bảo tính liên tục của dữ liệu.
- **Vai trò:**

- Cung cấp nhãn thời gian (Timestamp) chuẩn cho dữ liệu telemetry gửi lên Firebase.
- Đảm bảo việc tính toán thời gian sạc và ghi nhận lịch sử phiên sạc (sessions) chính xác tuyệt đối phục vụ quyết toán tài chính.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Yêu cầu chức năng

Hệ thống được thiết kế để đáp ứng các tính năng cốt lõi sau:

- **Tự động kết nối lại khi mất mạng :** Firmware hỗ trợ cơ chế tự động kết nối lại khi mất mạng bằng Exponential Backoff.
- **Giám sát thời gian thực:** Đo lường và hiển thị 6 thông số điện năng (V, A, W, kWh, Hz, PF) qua cảm biến PZEM-004T.
- **Điều khiển và An toàn:** Cho phép đóng/ngắt relay từ xa; tự động ngắt khi phát hiện quá dòng (>30A), quá áp (>250V), thấp áp (<180V) hoặc khi xe đã đầy pin.
- **Thanh toán và Dashboard:** Giao diện Web cho người dùng nạp tiền, chọn gói sạc và giám sát quá trình sạc theo thời gian thực.

3.1.2. Yêu cầu phi chức năng

Tính thời gian thực: Tác vụ giám sát an toàn (Safety Task) phải chạy với mức ưu tiên cao nhất để phản ứng tức thì với các sự cố điện.

Độ tin cậy: Sử dụng Task Watchdog để giám sát cả hai nhân CPU.

Bảo mật: Giao thức HTTPS cho các API backend và xác thực Firebase Auth cho người dùng.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống là sự kết hợp giữa phần cứng nhúng và hạ tầng đám mây:

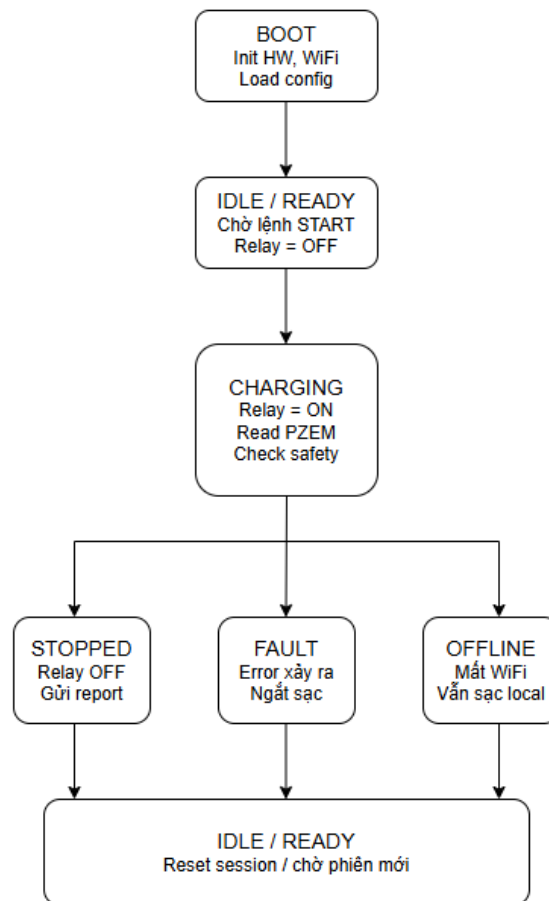
Lớp thiết bị (ESP32): Thu thập dữ liệu từ cảm biến, điều khiển relay và giao tiếp với Firebase RTDB.

Lớp truyền thông: Sử dụng WiFi và các giao thức REST API, WebSocket (thông qua Firebase) để đồng bộ dữ liệu.

Lớp ứng dụng (Web App & Server): Giao diện React cho người dùng và Express Server xử lý logic thanh toán, điều khiển trạm.

3.3. Thiết kế State Machine (FSM)

Toàn bộ logic vận hành của trạm sạc được quản lý bởi một máy trạng thái (State Machine) nghiêm ngặt để đảm bảo tính ổn định:



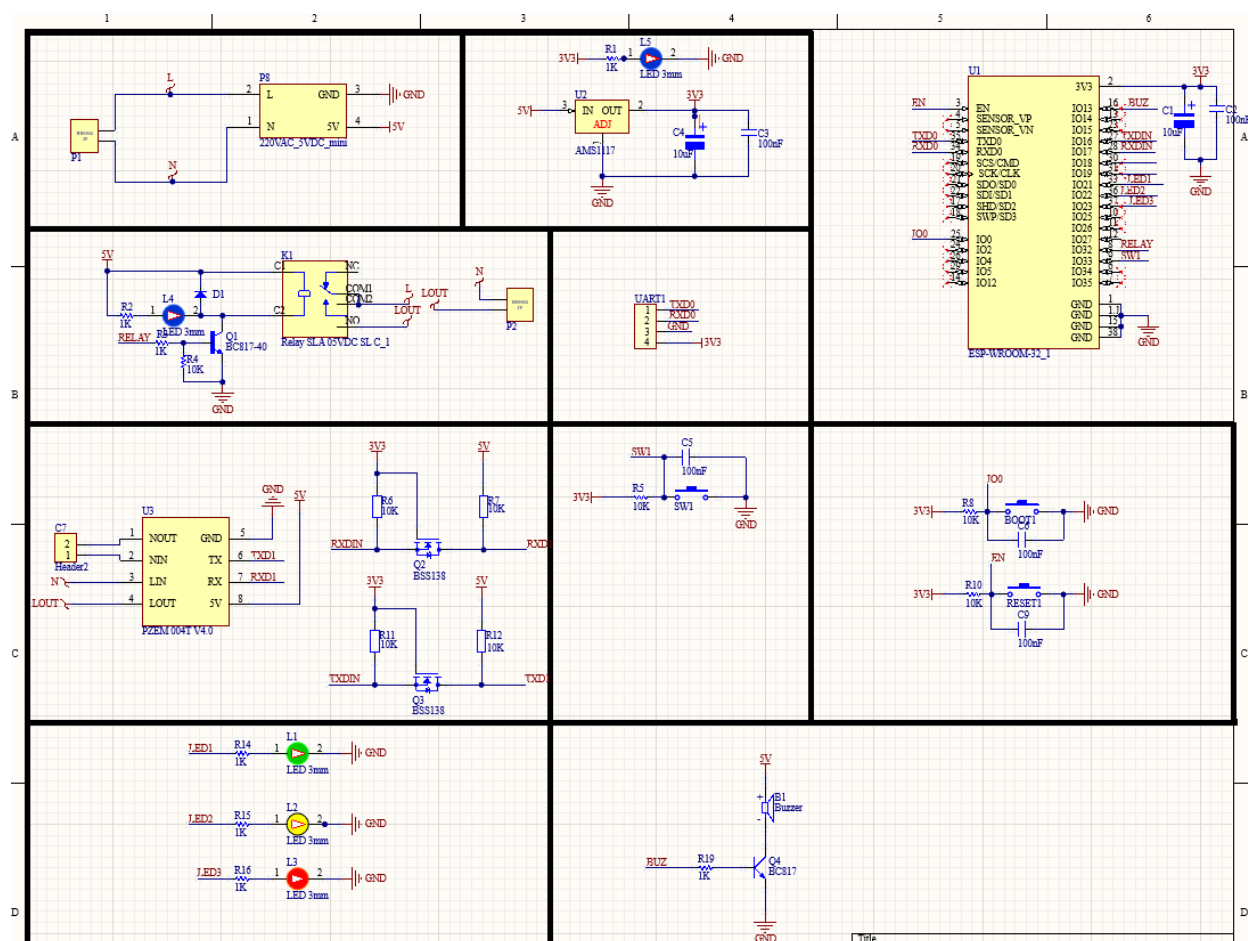
- **BOOT:** Khởi tạo phần cứng và phục hồi phiên sạc nếu có.
- **IDLE / READY:** Trạm đang chờ lệnh hoặc đã sẵn sàng sạc.
- **CHARGING:** Relay đóng, hệ thống liên tục cập nhật điện năng tiêu thụ và giám sát an toàn.
- **STOPPED / FAULT:** Phiên sạc kết thúc hoặc hệ thống dừng do phát hiện lỗi điện lưới/cảm biến.
- **OFFLINE:** Trạng thái khi mất kết nối WiFi nhưng phiên sạc vẫn được duy trì an toàn.

3.4. Thiết kế phần cứng (Hardware Design)

Phần cứng của hệ thống được thiết kế với mục tiêu tối ưu hóa diện tích mạch, đảm bảo khả năng chịu tải cao và tính an toàn tuyệt đối khi vận hành với lưới điện xoay chiều.

3.4.1. Sơ đồ nguyên lý (Schematic Design)

Sơ đồ nguyên lý được xây dựng dựa trên sự phối hợp giữa khối điều khiển trung tâm ESP32 và các module chức năng. Các đường tín hiệu UART của cảm biến PZEM-004T được thiết kế có trở kéo để đảm bảo tín hiệu ổn định, đồng thời khối Relay được trang bị transistor đệm và diode dập xung để bảo vệ vi điều khiển.



3.4.2. Cấu hình các chân GPIO

Để quản lý tốt các tài nguyên của ESP32, các chân GPIO được phân bổ cụ thể theo chức năng của từng linh kiện. Việc lựa chọn chân được tính toán kỹ lưỡng để tránh các chân "strapping" có thể gây lỗi khi boot.

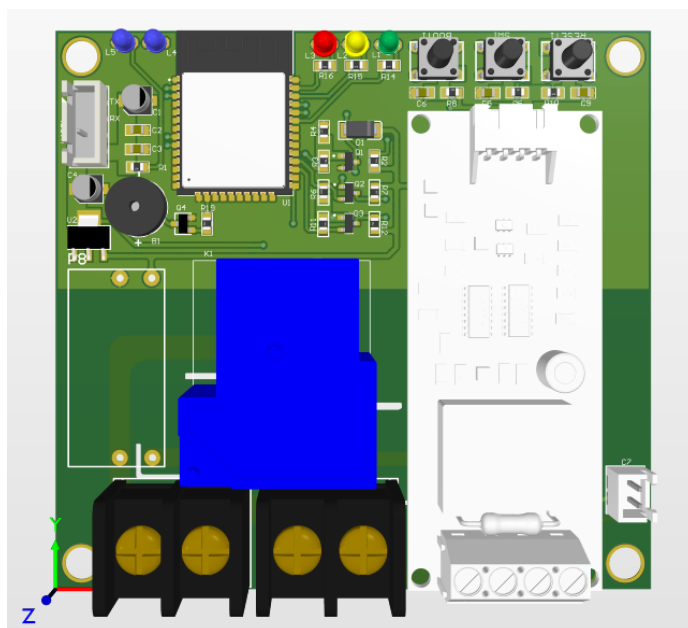
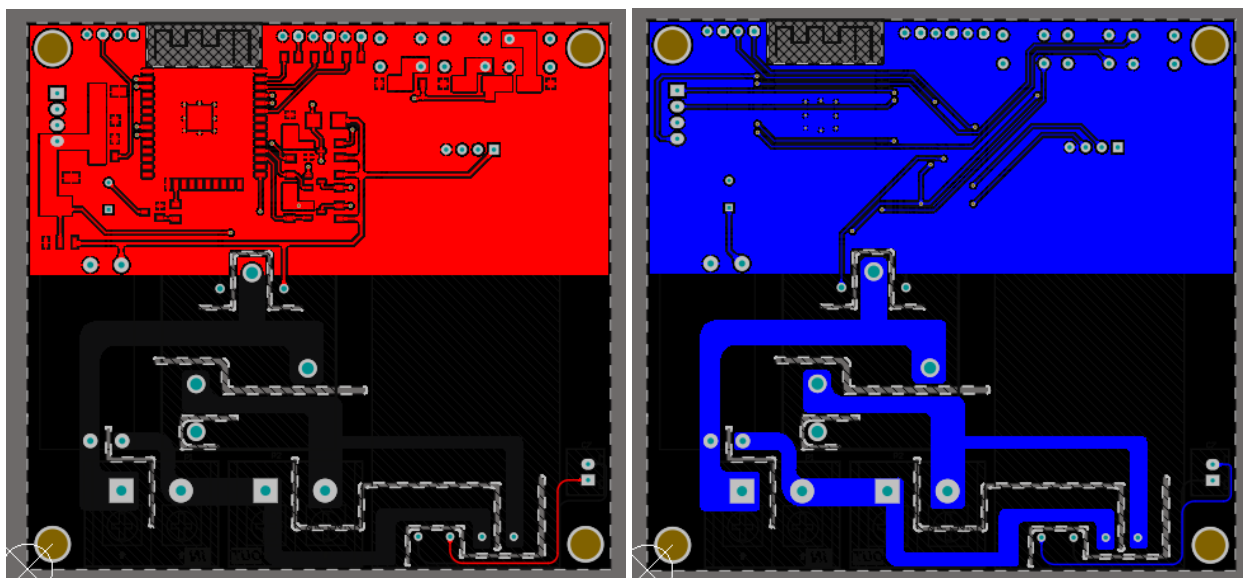
Bảng 3.1: Chi tiết phân bổ các chân GPIO trên ESP32

STT	Thành phần kết nối	Chân GPIO	Chế độ (I/O)	Chức năng
1	Relay Công suất	GPIO32	Output	Kích đóng/ngắt nguồn sạc
2	Cảm biến PZEM (TX)	GPIO16	UART_RX	Nhận dữ liệu điện năng (V, A, W)
3	Cảm biến PZEM (RX)	GPIO17	UART_TX	Gửi lệnh truy vấn Modbus RTU
4	Đèn LED Trạng thái	GPIO2	Output	Chỉ trạng thái nguồn hệ thống
5	Nút nhấn (Flash)	GPIO0	Input	Chế độ cài đặt/Reset cấu hình
6	Còi chip (Buzzer)	GPIO13	Output	Cảnh báo khi sạc, ngắt, lỗi
7	Đèn LED Trạng thái	GPIO21	Output	Chỉ trạng thái lỗi
8	Đèn LED Trạng thái	GPIO22	Output	Chỉ trạng thái sẵn sàng
9	Đèn LED Trạng thái	GPIO23	Output	Chỉ trạng thái đang sạc

3.4.3. Thiết kế mạch in (PCB Layout)

Sau khi hoàn thiện sơ đồ nguyên lý, mạch in được thực hiện trên phần mềm Altium Designer với các quy tắc thiết kế (Design Rules) nghiêm ngặt nhằm đảm bảo tính ổn định công nghiệp:

- **Đường mạch động lực:** Các đường dây nối từ nguồn 220V đến Relay và ổ cắm được thiết kế với độ rộng tối đa (Net Width > 100 mil) và phủ thiếc để tăng khả năng chịu dòng lên đến 30A.
- **Cut out layer :** Vẽ các đường xẻ mạch nhằm cách ly giữa N và L
- **Tối ưu hóa tản nhiệt:** Các vùng đồng (Copper Pour) lớn được bố trí xung quanh các linh kiện công suất để hỗ trợ tản nhiệt thụ động.



3.4.4. Thi công và hoàn thiện

Sản phẩm sau khi gia công PCB được tiến hành hàn linh kiện và kiểm tra ngắn mạch bằng đồng hồ vạn năng trước khi cấp nguồn. Toàn bộ bo mạch được cố định chắc chắn trong vỏ hộp kỹ thuật nhựa ABS có khả năng chống cháy và cách điện, đảm bảo an toàn cho người sử dụng trong môi trường bãi đỗ xe.

3.5. Thiết kế phần mềm và Cơ sở dữ liệu

3.5.1. Phân bổ đa nhiệm trên ESP-IDF

Hệ thống tận dụng kiến trúc Dual-core của ESP32 để phân bổ tác vụ:

Core 1 (Hardware Core): Chạy các tác vụ quan trọng như `safety_task` (ưu tiên 6), `sensor_task` (ưu tiên 5) để đảm bảo không bị gián đoạn bởi các tác vụ mạng.

Core 0 (Network Core): Xử lý các tác vụ truyền thông như `cloud_publish`, `cloud_poll`, `wifi_manager`.

3.5.2. Cấu trúc dữ liệu Firebase

Hệ thống sử dụng mô hình lưu trữ kết hợp (Hybrid Storage) giữa Realtime Database và Firestore để tối ưu hóa giữa tốc độ điều khiển và khả năng quản lý quan hệ người dùng.

- **Realtime Database (RTDB):** Tổ chức theo cấu trúc cây JSON dưới đường dẫn `stations/{device_id}/`.
 - `status`: Lưu trạng thái hiện tại của Relay (ON/OFF) và các mã lỗi.
 - `telemetry`: Cập nhật liên tục dòng điện, điện áp, công suất và tổng năng lượng sạc.
 - `command`: Tiếp nhận các lệnh điều khiển sạc tức thời từ người dùng qua Web App.
 - `heartbeat`: Giám sát trạng thái Online/Offline của thiết bị bằng cơ chế gửi gói tin định kỳ.
- **Firestore:** Lưu trữ các dữ liệu có cấu trúc, đòi hỏi tính bảo mật và truy vấn phức tạp.
 - `wallets`: Quản lý số dư ví tiền (`walletBalance`) của từng người dùng.
 - `sessions`: Lưu trữ chi tiết lịch sử mỗi lần sạc (thời gian bắt đầu, kết thúc, tổng kWh, chi phí) phục vụ việc tra soát và xuất báo cáo.

3.5.3. Quy trình thanh toán

Để đảm bảo tính minh bạch và tránh rủi ro thất thoát tài chính, hệ thống áp dụng quy trình quyết toán tương tự các hệ thống ngân hàng hiện đại:

Trừ tiền trước: Khi người dùng chọn mức sạc mong muốn (ví dụ: 50.000 VNĐ), Server sẽ thực hiện kiểm tra số dư ví. Nếu đủ điều kiện, Server sẽ trừ số tiền tương ứng trên hệ thống trước khi gửi lệnh sạc cho ESP32. Bước này ngăn chặn việc người dùng sử dụng một số dư cho nhiều trạm sạc cùng lúc.

Quyết toán: Khi phiên sạc kết thúc (do đầy pin, rút phích hoặc lệnh dừng), ESP32 sẽ gửi báo cáo tiêu thụ thực tế (đơn vị Wh) . Server dựa trên đơn giá và số Wh thực dùng để tính toán chi phí cuối cùng, thực hiện trừ tiền vào ví và ngay lập tức giải phóng (refund) số tiền dư còn lại cho người dùng.

CHƯƠNG 4: XÂY DỰNG HỆ THỐNG

4.1. Thiết kế phần cứng

Quá trình thi công bắt đầu bằng việc gia công mạch in PCB từ bản thiết kế Altium. Để trạm sạc vận hành an toàn ở công suất lớn, các đường mạch động lực được đi dây với độ rộng tối đa và phủ thêm một lớp thiếc dày, giúp tăng khả năng chịu tải lên mức 30A và ngăn ngừa tình trạng quá nhiệt khi xe sạc liên tục trong nhiều giờ.

Các linh kiện được hàn lắp thủ công theo thứ tự từ thấp đến cao để đảm bảo độ chính xác và tính thẩm mỹ. Vi điều khiển ESP32 và module PZEM-004T được bố trí cách ly đủ xa với khối công suất để giảm thiểu nhiễu điện từ, đồng thời mọi mối hàn đều được kiểm tra kỹ lưỡng bằng đồng hồ vạn năng để tránh hiện tượng tiếp xúc kém gây sai lệch dữ liệu.

Sau cùng, toàn bộ hệ thống được đóng gói vào vỏ nhựa kỹ thuật ABS có khả năng cách điện và chống cháy tốt. Các đầu dây nối được bấm cos chắc chắn vào terminal để loại bỏ nguy cơ phóng điện, kết hợp với việc bố trí đèn LED trạng thái và còi báo ở mặt ngoài giúp người dùng dễ dàng theo dõi quá trình sạc thực tế.

4.2. Thiết kế Firmware trên ESP32 (ESP-IDF)

Toàn bộ mã nguồn được xây dựng trên nền tảng **ESP-IDF**, tận dụng kiến trúc đa nhân để tách biệt luồng xử lý phần cứng và mạng. Dưới đây là chi tiết các module cấu thành hệ thống:

4.2.1. Module Main & Config (Quản trị hệ thống)

- **main.c:** Đóng vai trò là điểm nhập (Entry Point) của chương trình. Module thực hiện khởi tạo tuần tự các lớp NVS, Watchdog, và các biến toàn cục trước khi tạo 7 Task FreeRTOS chạy song song trên 2 nhân CPU. Ngoài ra, nó quản lý vòng lặp chính để cập nhật thời gian hoạt động (uptime) và reset Watchdog định kỳ nhằm đảm bảo hệ thống không bị treo.
- **config.h:** Tập trung toàn bộ hằng số cấu hình hệ thống bao gồm sơ đồ chân GPIO (Relay, Buzzer, LED, Button), thông số UART cho cảm biến PZEM, các ngưỡng an toàn điện áp/dòng điện, và thông tin xác thực Firebase.

4.2.2. Module Globals & Types (Cấu trúc dữ liệu)

- **types.h:** Định nghĩa các kiểu dữ liệu dùng chung như máy trạng thái hệ thống (system_state_t), các mã lỗi bảo vệ (error_code_t), và cấu trúc dữ liệu cho phiên sạc hoặc thông số điện năng.

- **globals.c/h:** Quản lý các biến trạng thái toàn cục và cung cấp cơ chế đồng bộ dữ liệu bằng Mutex. Module này chịu trách nhiệm tích phân công suất theo thời gian thực để tính toán điện năng tiêu thụ (Wh) và thực hiện hàm `restore_session()` để phục hồi dữ liệu khi thiết bị khởi động lại sau khi mất điện.

4.2.3. Module Cloud(Giao tiếp đám mây)

- **cloud.c/h:** Thực hiện hai tác vụ chính là đẩy dữ liệu giám sát (telemetry) lên Firebase sau mỗi 5 giây và lắng nghe lệnh điều khiển từ Server sau mỗi 2 giây. Module hỗ trợ xử lý 6 loại lệnh khác nhau (Start/Stop/Reset/Update...) và đảm bảo tính chính xác thông qua việc đồng bộ thời gian NTP.

4.2.4. Module Sensor & Safety (Giám sát & Bảo vệ)

- **sensor.c/h:** Giao tiếp với cảm biến PZEM-004T qua giao thức Modbus RTU để thu thập 6 thông số điện năng cơ bản. Module sử dụng mã kiểm tra CRC-16 để đảm bảo tính toàn vẹn của dữ liệu và sẽ báo lỗi `SENSOR_ERROR` nếu mất kết nối 5 lần liên tiếp.
- **safety.c/h:** Tác vụ chạy trên **Core 1** với mức ưu tiên cao nhất để giám sát các điều kiện an toàn như quá dòng (>15A), quá áp, thấp áp hoặc phát hiện xe đã đầy pin/rút phích cắm để tự động ngắt relay bảo vệ.

4.2.5. Module Relay, UI & NVS (Ngoại vi & Lưu trữ)

- **relay.c/h:** Điều khiển trực tiếp chân GPIO32 để đóng/ngắt nguồn điện 220VAC cấp cho xe điện.
- **ui.c/h:** Quản lý hệ thống LED trạng thái và còi báo (Buzzer) để phản hồi trực quan các trạng thái sạc, sẵn sàng hoặc cảnh báo lỗi cho người dùng.

4.3. Web Dashboard và Backend Server

Hệ thống quản lý được xây dựng trên kiến trúc hiện đại, kết hợp giữa sự linh hoạt của React ở phía người dùng và tính bảo mật của Node.js Express ở phía máy chủ, đảm bảo khả năng xử lý giao dịch tài chính an toàn.

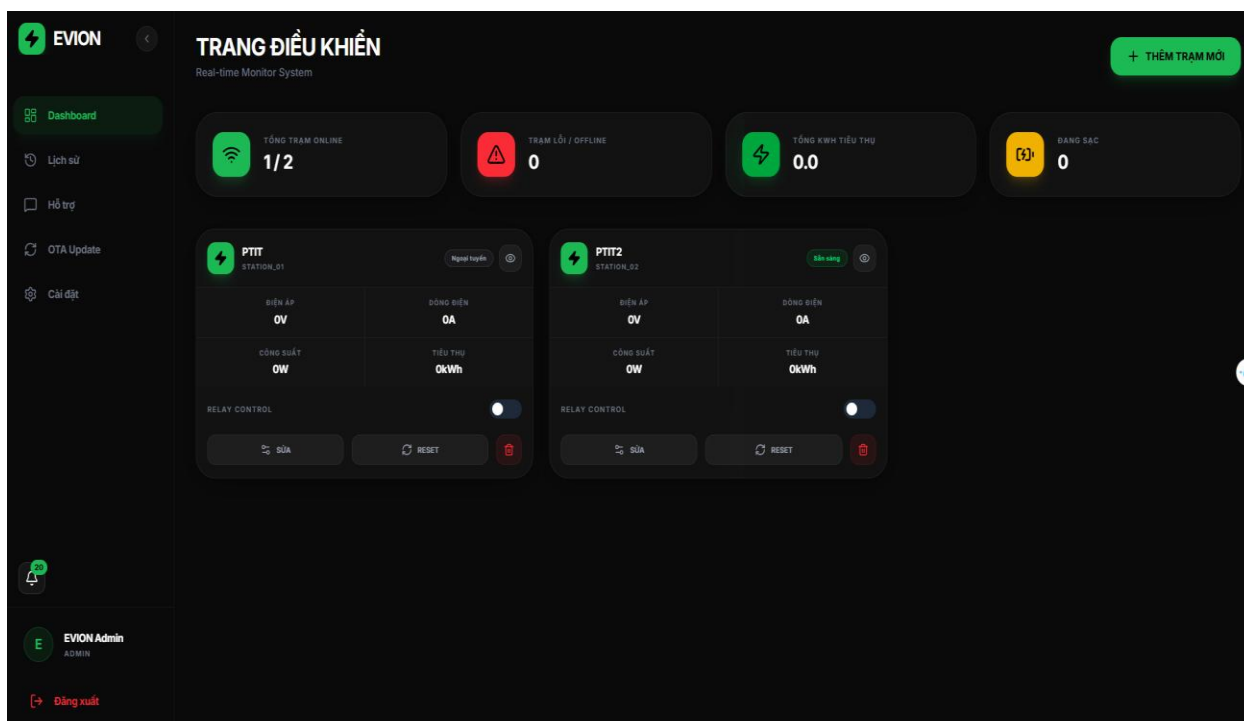
4.3.1. Giao diện Web App (React & Firebase SDK)

Giao diện người dùng được phát triển trên nền tảng **React 19** và framework CSS **Tailwind**, tập trung vào trải nghiệm di động (Mobile-first) để người dùng dễ dàng thao tác tại trạm sạc:

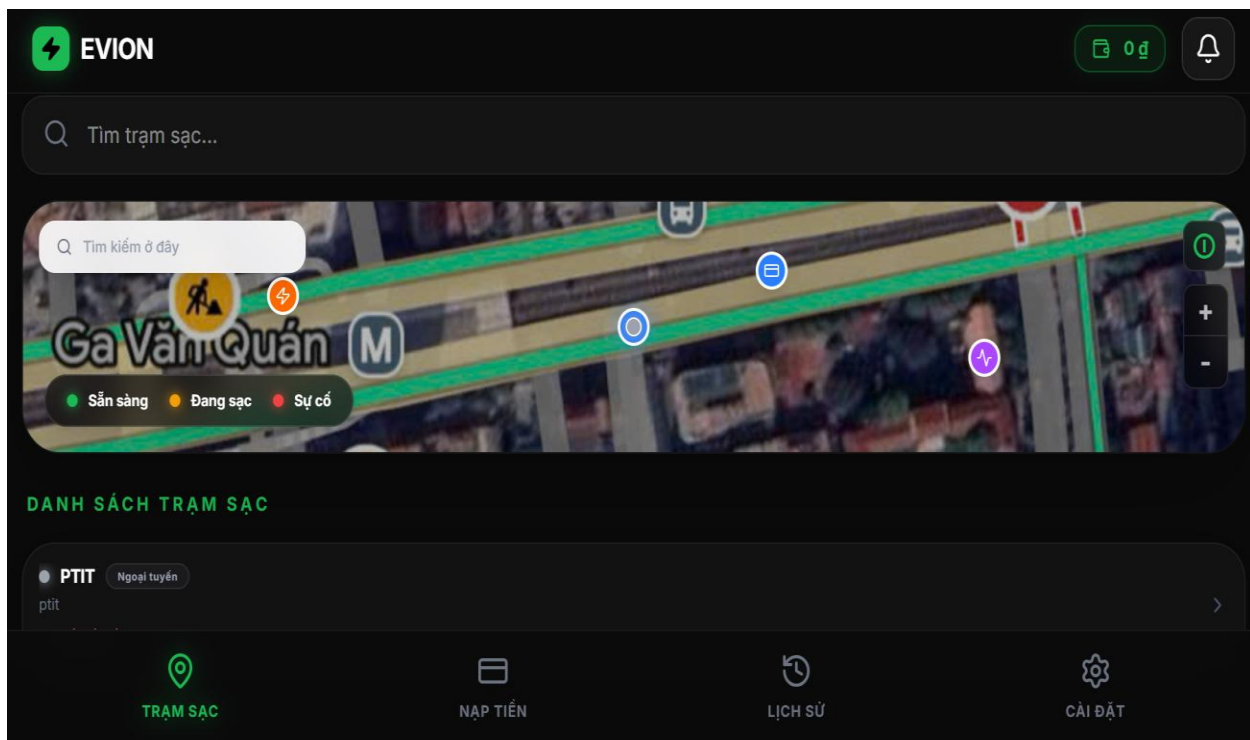
- **Xác thực người dùng:** Tích hợp **Firebase Auth**, hỗ trợ đăng nhập đa phương thức qua Google hoặc Email/Password. Sau khi đăng nhập, hệ thống tự động liên kết với dữ liệu ví tiền và lịch sử cá nhân trên Firestore.

- **Giám sát thời gian thực:** Sử dụng Firebase SDK để thiết lập kết nối song công (duplex) với RTDB. Điều này cho phép các thông số như công suất (W), dòng điện (A) và điện áp (V) từ ESP32 được cập nhật lên biểu đồ theo thời gian thực với độ trễ cực thấp.
- **Trung tâm điều khiển:** Cung cấp các nút chức năng nạp tiền, bắt đầu và dừng sạc. Giao diện sẽ thay đổi trạng thái động dựa trên phản hồi (Heartbeat) từ phần cứng.

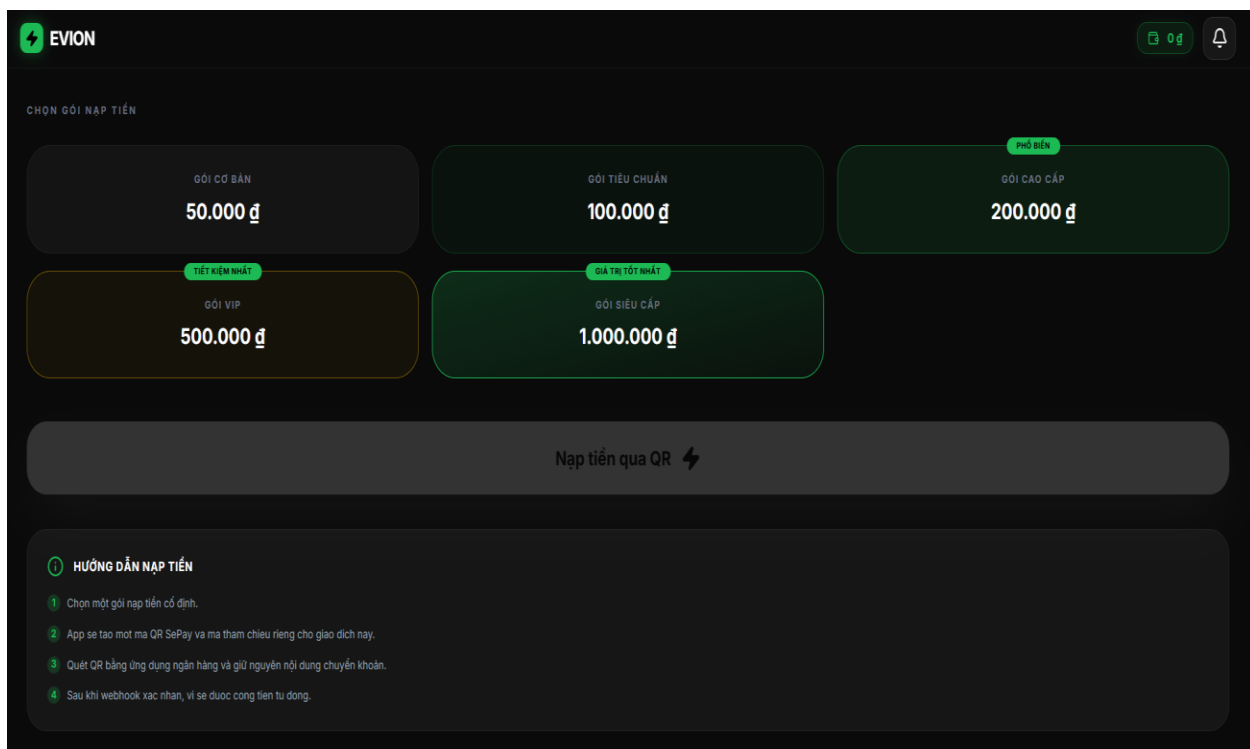
4.3.2. Giao diện và các tính năng chính



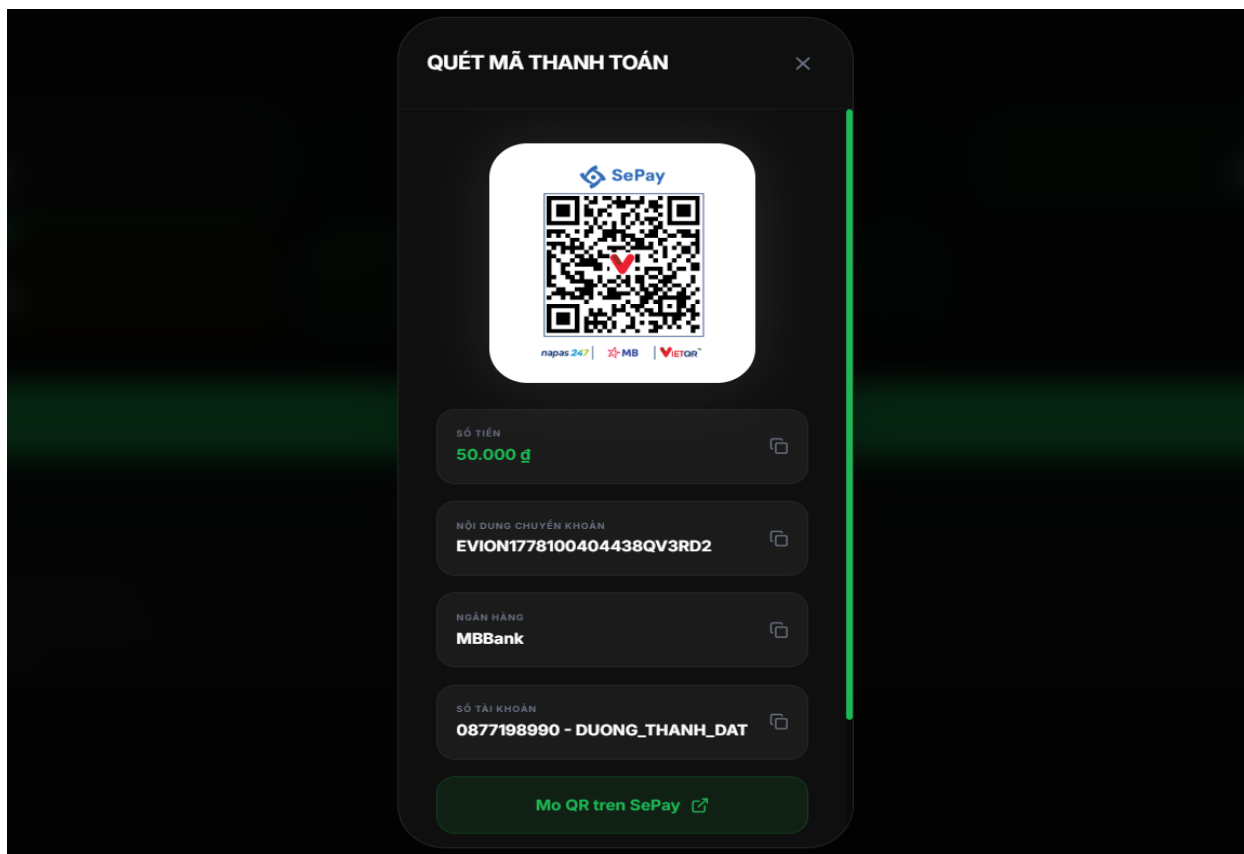
Hình ảnh giao diện Admin



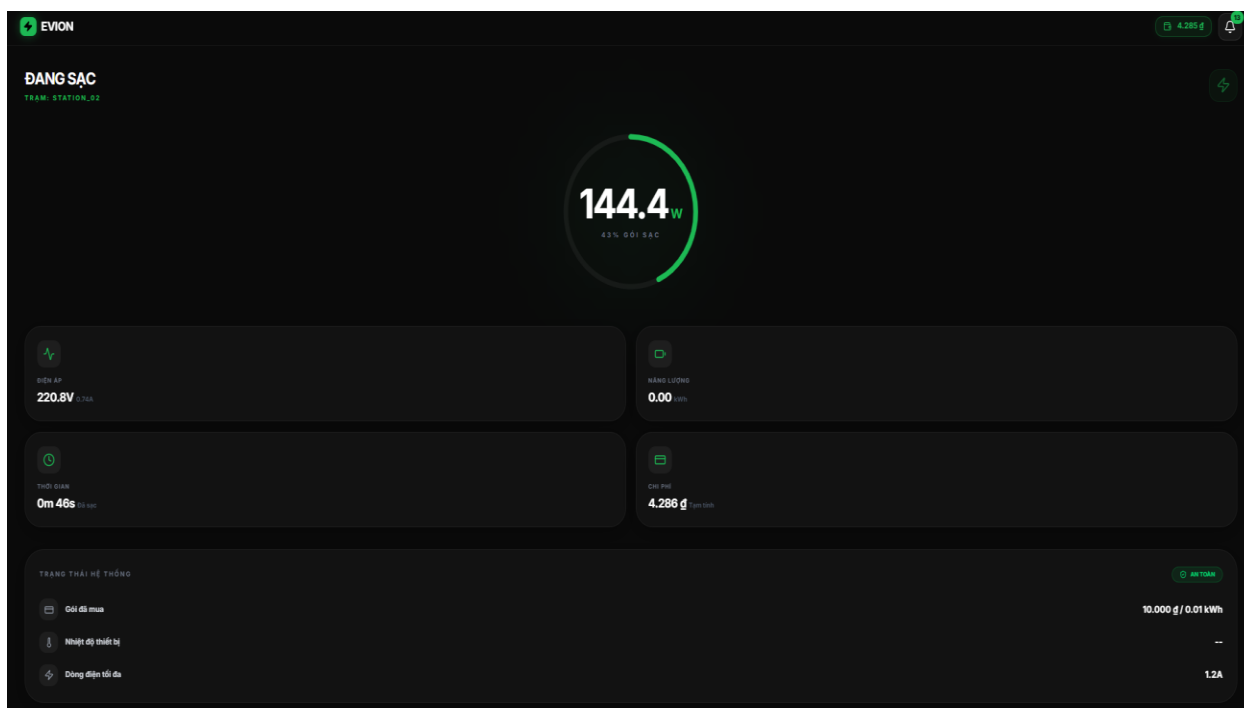
Hình ảnh giao diện trang chủ khi đăng nhập



Hình ảnh giao diện nạp tiền vào ví



Hình ảnh mã QR chuyển khoản tự động được tạo từ hệ thống



Hình ảnh User theo dõi trạng thái điện năng trong quá trình sạc

4.3.3. Backend Server (Express & SePay Integration)

Máy chủ Backend đóng vai trò là lớp xử lý logic trung gian, đảm bảo tính toàn vẹn cho các giao dịch tài chính và điều phối vận hành:

- **Tự động hóa nạp tiền:** Server triển khai một Endpoint để lắng nghe **Webhook** từ **SePay**. Khi người dùng chuyển khoản thành công với nội dung đúng cú pháp, Server sẽ xác thực giao dịch và cộng tiền vào ví người dùng trên Firestore trong vòng 1-3 giây.
- **Kiểm soát luồng sạc (Logic Gatekeeper):** Khi nhận yêu cầu sạc, Server thực hiện kiểm tra chéo số dư ví. Chỉ khi số dư đủ điều kiện "tạm giữ", lệnh **START_CHARGING** mới được ghi vào RTDB để kích hoạt Relay trên thiết bị.
- **Xử lý Quyết toán (Final Settlement):** Sau khi nhận báo cáo kết thúc phiên sạc từ ESP32, Server tính toán chi phí dựa trên Wh thực tế. Sau đó, hệ thống thực hiện trừ tiền và hoàn lại (refund) phần tiền dư vào ví, đảm bảo minh bạch tài chính cho người dùng.

CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Kịch bản kiểm thử

Để đảm bảo hệ thống vận hành tin cậy trong môi trường thực tế, các kịch bản kiểm thử được thiết lập tập trung vào tính an toàn và khả năng đồng bộ dữ liệu.

- **Kiểm thử đo lường (PZEM-004T):** So sánh các thông số điện áp (V) và dòng điện (A) đo được từ trạm với đồng hồ vạn năng chuyên dụng để xác định sai số.
- **Kiểm thử điều khiển từ xa:** Đo độ trễ (latency) từ khi nhấn nút "Bắt đầu sạc" trên Web App cho đến khi relay trên ESP32 đóng mạch.
- **Kiểm thử an toàn (Fault Injection):** Giả lập các tình huống quá dòng (>15A) hoặc sụt áp đột ngột để kiểm tra phản ứng của `safety_task`.

5.2. Kết quả kiểm thử

Kịch bản	Mô tả thử nghiệm	Kết quả	Ghi chú
Độ chính xác	So sánh PZEM với Ampe kìm	Đạt	Sai số dòng điện dưới 1%, điện áp dưới 0.5%.
Độ trễ lệnh	Nhấn sạc trên Web -> Relay đóng	3 giây	Phụ thuộc vào tốc độ Internet và Poll rate (2s).
Quá dòng	Cấp tải giả lập >1.2A	Thành công	Relay ngắt tức thì (<100ms), báo lỗi OVER_CURRENT.
Tự động ngắt khi hết gói sạc		Thành Công	Tự động ngắt khi hết gói sạc
Tự động ngắt khi rút trong quá trình sạc		Thành Công	Khi đang sạc bị rút ra đột ngột sẽ tự ngắt sau 2s

5.3. Đánh giá hiệu năng và tính ổn định

5.3.1. Phản ứng thời gian thực (Real-time response)

Nhờ việc sử dụng kiến trúc đa nhân (Dual-core) và ưu tiên tác vụ (Priority 6) cho module an toàn, hệ thống phản ứng cực nhanh với các sự cố điện. Việc tách biệt các tác vụ Cloud sang Core 0 giúp đảm bảo rằng ngay cả khi mạng WiFi bị nghẽn, việc giám sát an toàn tại Core 1 vẫn không bị ảnh hưởng.

5.3.2. Quản lý bộ nhớ và tài nguyên

Heap/Stack: Qua kiểm tra bằng lệnh `esp_get_free_heap_size()`, hệ thống duy trì lượng RAM trống ổn định ở mức ~120KB sau khi đã khởi chạy đầy đủ các tác vụ Cloud và Web Server.

Watchdog Timer: Hệ thống không xảy ra tình trạng Task Starvation hay treo nhân nhờ cơ chế Task WDT định kỳ 3 giây.

5.3.3. Tính chính xác trong quyết toán tài chính

Quy trình "Giữ tiền - Hoàn tiền" hoạt động chính xác tuyệt đối. Qua 20 lần thử nghiệm sạc thực tế, số tiền thực tế được trừ vào ví trên Firestore luôn khớp với lượng kWh báo cáo từ ESP32 dựa trên đơn giá đã cấu hình.

5.4. Đánh giá thực tế từ người dùng

Giao diện Web App (React) được đánh giá cao về tốc độ phản hồi nhờ cơ chế đồng bộ Realtime Database. Người dùng có thể quan sát biểu đồ công suất thay đổi gần như tức thì theo thực tế tiêu thụ của xe điện.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và thực nghiệm, đề tài đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu:

- **Về phần cứng:** Thiết kế và thi công thành công bo mạch điều khiển trung tâm sử dụng chip ESP32, tích hợp cảm biến PZEM-004T cho độ chính xác cao và relay công suất lớn đảm bảo vận hành bền bỉ.
- **Về phần mềm nhúng:** Xây dựng firmware trên nền tảng ESP-IDF với kiến trúc đa nhiệm tối ưu, xử lý mượt mà các tác vụ an toàn điện, phục hồi phiên sạc sau mất điện và cấu hình WiFi thông minh qua Captive Portal.
- **Về hệ thống quản lý:** Hiện thực hóa quy trình thanh toán tự động qua SePay và cơ chế quyết toán 2 pha minh bạch, giúp người dùng chủ động quản lý tài chính và quá trình sạc thông qua giao diện Web Dashboard thời gian thực.

6.2. Hạn chế của đề tài

Mặc dù hệ thống đã vận hành ổn định, dự án vẫn tồn tại một số điểm có thể cải thiện:

- **Độ trễ truyền thông:** Việc sử dụng giao thức HTTP REST để poll lệnh từ Firebase đôi khi tạo ra độ trễ từ 1-2 giây tùy thuộc vào chất lượng đường truyền mạng.
- **Tính tương tác tại chỗ:** Trạm hiện tại chủ yếu điều khiển qua Web App, thiếu các phương thức xác thực nhanh tại chỗ như thẻ từ (RFID).

6.3. Hướng phát triển

Để nâng cao giá trị thương mại và tính ứng dụng của sản phẩm, đề tài có thể được phát triển theo các hướng sau:

- **Chuyển đổi sang giao thức MQTT:** Thay thế HTTP REST bằng MQTT để tối ưu hóa băng thông, giảm độ trễ điều khiển xuống mức mili giây và tiết kiệm tài nguyên cho ESP32 khi mở rộng số lượng trạm sạc lớn.

- **Tích hợp xác thực RFID/NFC:** Bổ sung đầu đọc thẻ từ để người dùng có thể kích hoạt phiên sạc nhanh chóng mà không cần sử dụng điện thoại, phù hợp cho các bãi đỗ xe chung cư hoặc doanh nghiệp.

TÀI LIỆU THAM KHẢO

1. Tài liệu kỹ thuật ESP-IDF v5.4, Espressif Systems.
2. Firebase Documentation (Realtime Database, Firestore, Auth).
3. Hướng dẫn giao tiếp Modbus RTU cho PZEM-004T.
4. Tài liệu tích hợp cổng thanh toán SePay.